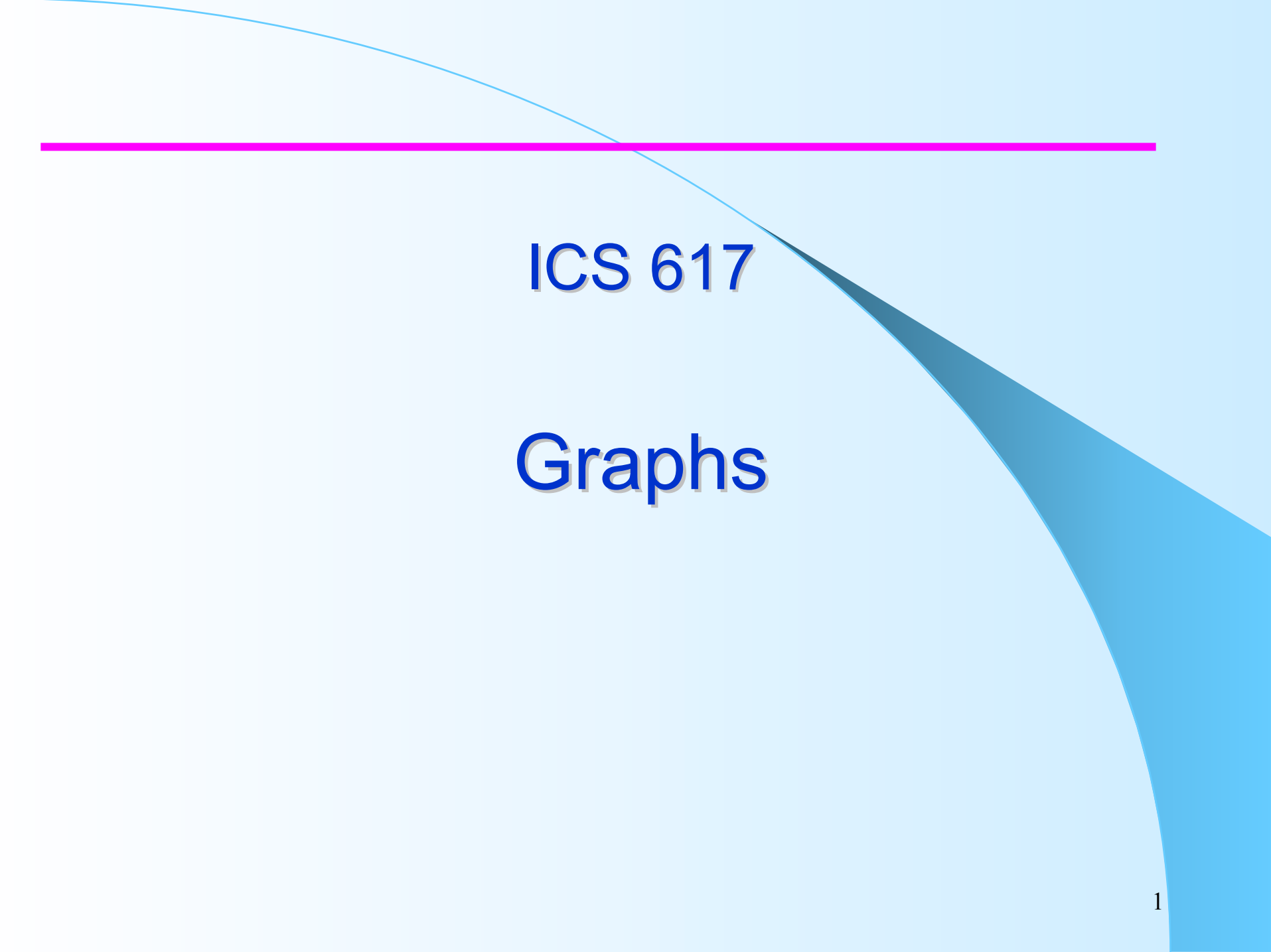




ICS 617

Graphs

Introduction

- ❑ A graph consists of vertices (nodes) and edges connecting the vertices ($G=(V,E)$)
- ❑ Graphs usually have a shape dictated by physical problems
- ❑ For instance nodes may represent cities, while edges may represent airline flight routes between cities
- ❑ Another more abstract example is a graph representing individual tasks necessary to complete a project
- ✓ In the graph, nodes may represent tasks, while directed edges (with an arrow at one end) indicate which task must be complete before another.

Introduction

- ❑ Two vertices are said to be adjacent to one another if they are connected by a single edge
- ❑ In figure 1, vertices H and G are adjacent
- ❑ Adjacent vertices are also said to be neighbors

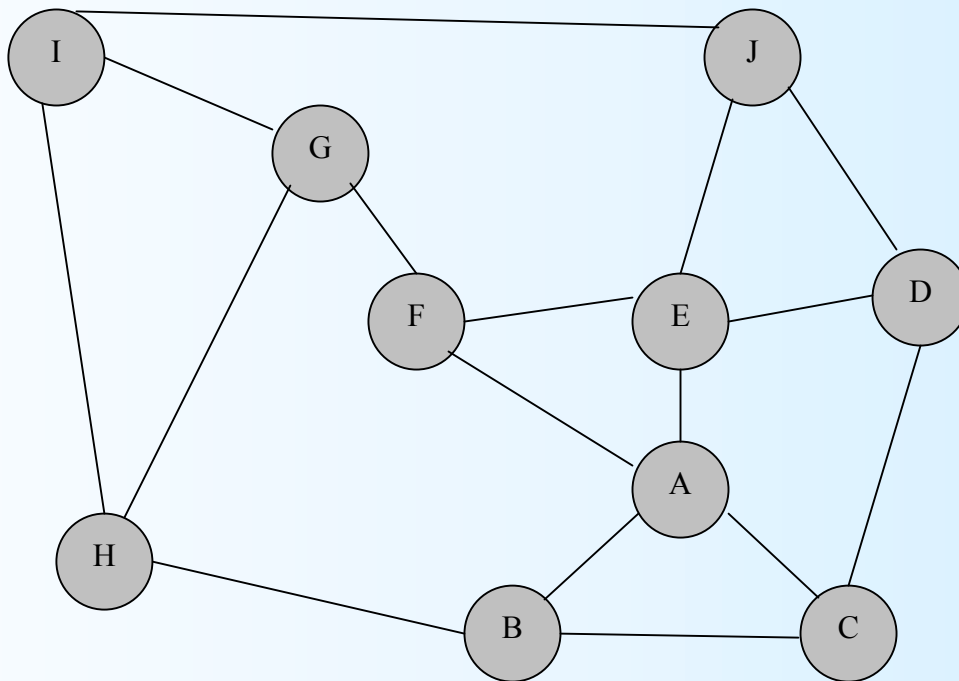


Fig 1: Graph

Introduction

- ❑ A path is a sequence of edges
- ✓ In figure 1, IGFE is a path
- ❑ There can be more than one path between two vertices
- ❑ A graph is said to be connected if there is at least one path from every vertex to every other vertex
- ❑ A non-connected graph consists of several connected components

Non-directed Graphs

- ❑ The edges for these graphs do not have a direction. One can go either way on them
- ❑ In figure 1, one can go from **G** to **I** or **I** to **G** on the same edge

Introduction

Directed Graphs

- ❑ Used to model situations in which one can go in only one direction along an edge, such as a one-way street
- ❑ Such a graph is said to be directed
- ❑ The allowed direction is typically shown with an arrow-head at the end of the edge
- ❑ In figure 2, vertex A is adjacent to C but not vice-versa

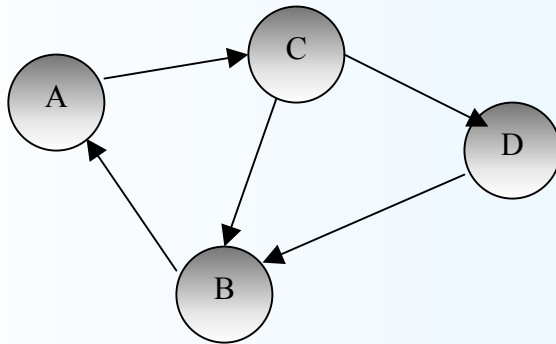


Fig 2: Directed Graph

Introduction

Weighted Graphs

- ❑ In some graphs, edges are given a weight
- ❑ Such a weight can be used to represent some property of the connection between two vertices e.g.
 - ✓ Physical distance
 - ✓ Cost of the route
 - ✓ Time taken to travel on the route

Representing A Graph In a Program

- ❑ We need to represent both the vertices and the edges
- ❑ **Nodes/Vertices** can be represent using an array.
- ❑ In this sense, nodes have to be numbered from node 0, to node $n-1$ (where n is the number of nodes)
- ❑ Node 0 is stored at position 0 in the array, node 1 at position 1, until node $n-1$ at position n

Introduction

- ❑ Each node will be represented with a structure that is appropriate to store the information associated with a node E.g.
 - ✓ A record /Structure (in C)
 - ✓ An object (object oriented languages)
 - ✓ A hash (in perl)
- ❑ There are two standard ways for representing edges
 - ✓ Adjacency Matrix
 - ✓ Adjacency list

Adjacency Matrix

- ❑ The adjacency matrix is a two dimensional array in which the elements indicate whether an edge is present between two vertices
- ❑ If a graph has n vertices, then the adjacency matrix is $n \times n$ array.

Introduction

- ❑ Entry i, j $j=0 \dots n-1$, $i=0 \dots n-1$ is 1 if there is an edge going from node i to node j . the entry is zero otherwise
- ❑ For a non-directed graph, entry $i, j = \text{entry } j, i$
- ❑ Entry i, i can be set as 0 or 1 as is convenient.

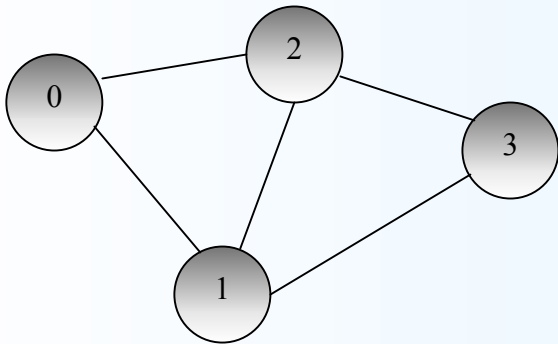


Fig 3: non-directed graph

	0	1	2	3
0	0	1	1	0
1	1	0	1	1
2	1	1	0	1
3	0	1	1	0

Fig 4: Adjacency matrix for figure 3

Introduction

Adjacency List

- ❑ The other way to represent edges is using an adjacency list
- ❑ An adjacency list is actually an array of lists
- ❑ The adjacency list is the preferred way to represent sparse graphs
- ❑ An adjacency list is defined using a one dimensional array.
- ❑ Each array position corresponds to a node of the graph.
- ❑ The entry for each position is a list, containing all vertices adjacent to that node.

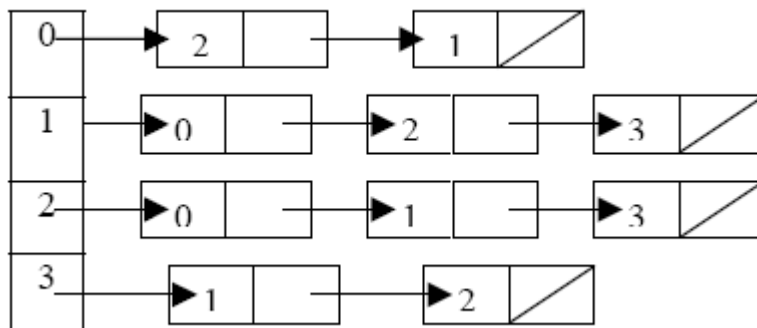


Fig 5: Adjacency list for figure 3

Introduction

- ❑ The adjacency list can easily be adapted to represent weighted graphs
- ✓ The weight for an edge is stored together with the corresponding vertex entry on the adjacency list associated with a node

Searches

- ❑ One of the most fundamental operations to perform on a graph is finding which vertices can be reached from a specified vertex
- ❑ There are two common approaches to searching. These are:
 - ✓ breath-first search
 - ✓ depth-first search

Introduction

Depth-First Search

- ❑ Uses a LIFO queue (e.g. a stack) to remember where it should proceed with the search
- ❑ This algorithm searched deeper into the graph wherever possible.
- ❑ Its proceeds as follows given the starting node *s*, and that pop returns null if the stack is empty. *c* is the current node

```
1) c:=s
2) Mark c as visited
3) If there is a node x adjacent to c not yet visited
4) Push c into the stack
5)     c:=x
6)     mark c as visited
7) Else
8)     c=pop item on top of stack
9) Repeat from 3 while c<>null
```

Introduction

Analysis of depth first search

- ❑ Time complexity is $O(n)$, because it usually needs to visit all the vertices
- ❑ Space complexity is order $d \times b$, where d is the depth of the farthest node it will need to visit from the start node, b is the maximum number of nodes adjacent to any node

Modifications

- ❑ The algorithm can be modified to show the visited nodes, and their distance from the start node
- ❑ This information is added to the vertices array.

Introduction

Breath-First Search

- ❑ Uses a FIFO queue to remember where it should proceed with the search
 - ❑ The algorithm first visits the nodes that are closest to the start node
 - ❑ Its proceeds as follows given the starting node s ,
 - ❑ Assume that dequeue returns *null* if the queue is empty and that the current node is v
- 1) **$v := s$**
 - 2) Mark **v** as visited
 - 3) For all **x** such that x is a child of v not yet visited
 - 4) mark **x** as visited
 - 5) add **x** to the back of the queue
 - 6) **$v :=$ first node in the queue (dequeue)**
 - 7) **Repeat from 3 while $v \neq \text{null}$**

Introduction

Analysis of Breath-First Search

- ❑ Given the above implementation of breath-first search, the time complexity is $O(n)$ to visit every node
- ❑ The space complexity is also $O(n)$ because every node will be inserted in the queue at least once

Introduction

Minimum Spanning Trees

- ❑ A connected graph has a path from every edge to every other edge
- ❑ There may be more than one path between any two nodes
- ❑ If we are given a graph that is connected, we might want to reduce it to a form where it is still connected but there is only one path between any two nodes
- ❑ The resulting graph is called a **minimum spanning tree** of the graph.
- ❑ There may be many minimum spanning trees for a given graph

Introduction

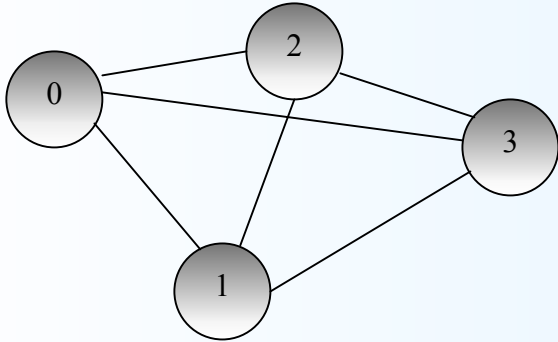


Fig 6: connected graph

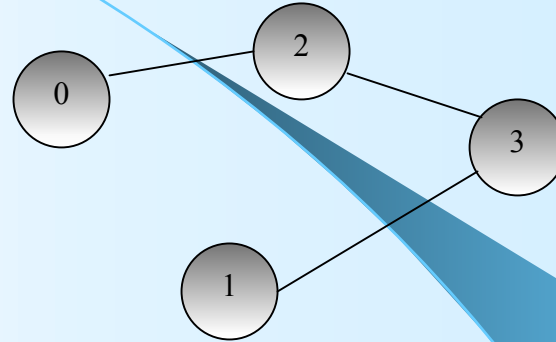


Fig 7: minimum spanning tree for figure 6

Introduction

Minimum Spanning Algorithm

- ❑ A minimum spanning tree can be constructed using a slight modification of the depth-first search algorithm
- ❑ We just need to record the edges we used to reach a node.
- ❑ This means that we will construct a new graph, containing only the edges we crossed in visiting nodes using a depth-first traverse

Introduction

Minimum-spanning tree algorithm

- ❑ Uses a LIFO queue (e.g. a stack) to remember where it should proceed with the search
- ❑ Construct an $n \times n$ matrix (m) and initialize its elements with zeros
- ❑ Given the starting node s , and that pop returns null if the stack is empty
 - 1) Current node $c := s$
 - 2) Mark c as visited
 - 3) **If** there is a node x adjacent to c not yet visited
 - 4) $m[c, x] = 1$
 - 5) $m[x, c] := 1$
 - 6) Push c into the stack
 - 7) $c := x$
 - 8) mark c as visited
 - 9) **Else**
 - 10) $c = \text{pop item on top of stack}$
 - 11) **Repeat from 3 while** $c \neq \text{null}$

Introduction

Topological Sorting With Directed Graphs

- ❑ Another operation that can be modeled with graphs
- ❑ Useful in situations where items or events must be arranged in a specific order
- ❑ The required order between items is usually represented as a directed graph
- ❑ If item i must come before item j , then, in the graph, there is a edge from node i to node j
- ❑ The following is a graph showing such an ordering for courses and their prerequisites at a university

Introduction

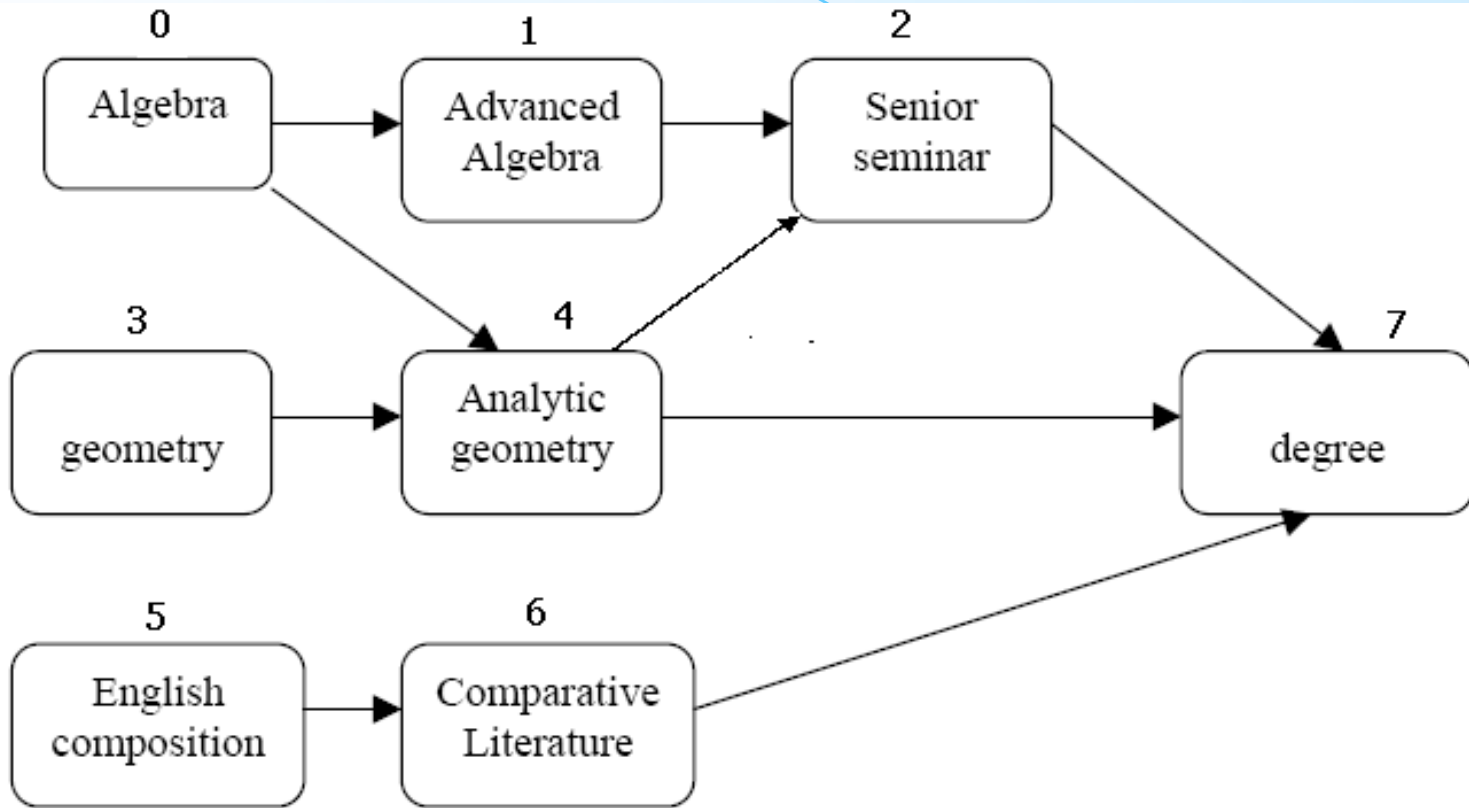


Fig 8: Prerequisites directed graph. For ease of processing, the nodes are numbered

Introduction

- ❑ The edges of the graph can be represented using an adjacency matrix
- ❑ The nodes can be represented using an array or a linked list as usual
- ❑ A topological sort is a listing of the elements in a directed graph such that the any node comes after its predecessors in the graph
- ❑ For instance for the graph in figure 8, a topologically sorted listing is as shown below
0, 5, 6, 3, 4, 1, 2, 7
- ❑ It shows a possible ordering of tasks

Introduction

Topological Sort Algorithm

- ❑ We have to define an array t that contains the topologically sorted nodes
- ❑ If the number of nodes is n , the size of t is also n
- 1) $v := \text{null}$
- 2) Find a node v that has no children (adjacent nodes)
- 3) **if** $v = \text{null}$
- 4) **DISPLAY** ERROR 'graph contains cycles'
- 5) **end**
- 6) $t[n-1] := v$
- 7) **delete** v from the list of nodes
- 8) **delete** any edge that contains v at one end
(delete row v and column v from the adjacency matrix)
- 9) **Repeat** from 2 **while** list of nodes is not empty
- 10) **Output** t

Introduction

- ❑ This algorithm works only for acyclic graphs.
- ❑ Cyclic graphs contain paths of length 1 or more that start from a node x and end at the same node x
- ❑ This algorithm is destructive in that the original graph is destroyed
- ❑ This can be prevented by making a copy of the graph and working with it